

From Faulty Memories to Corrected Actions: Dependency-Guided Rollback Repair for Memory-Augmented Agents

Caili Yu, Yiqi Wang*, Jiaqi Zhang, Yiqun Duan, Mingkai Zheng, Zhangkai Wu, Kaize Shi, Taotao Cai

* yiqi.wang.jennie@gmail.com

Abstract

Persistent memory lets language-model agents reuse information across sessions, but it also makes errors durable: a poisoned, stale, or misattributed record can alter reasoning, tool use, answers, and subsequent memory writes. Existing defenses mainly detect or delete suspicious memories, or revise the current response. Deleting the source leaves already propagated claims, actions, and derived memories active, whereas resetting the store or replaying the full trace destroys benign state and repeats unnecessary computation. We therefore formulate **post-failure memory recovery**: *given a failed execution and diagnosed faulty memories, recover both the answer and persistent state while retaining unaffected work*. Our **dependency-guided rollback repair** builds a typed memory-to-action graph from runtime provenance, traces explicit downstream dependencies, preserves candidates with independent trusted support, deactivates unsupported memory state, and selectively replays only answer-relevant affected computation. We evaluate this approach on a 150-case controlled benchmark spanning three tool-use domains and four memory failure types, and on a 50-case trajectory-derived stress test adapted from LongMemEval-V2. On the controlled benchmark, it achieves 85.3% recovery versus 77.3% for the best competing recovery method, removes all diagnosed faulty memories, preserves all benign memories, and requires only selective replay with modest LLM-call cost. On the adapted subset, it reaches 68.0% recovery versus 54.0% for the next best method, while also achieving the highest claim invalidation F1, 0.669 versus 0.603. Overall, the results do not imply uniformly better trace reconstruction, but show that dependency-guided rollback repair provides a strong recovery–cost trade-off while repairing faulty memory state and preserving benign memory.

1 Introduction

Persistent memory lets language model agents carry preferences, observations, and experience across sessions, enabling personalization and long-horizon behavior that a current prompt alone cannot support (Park et al. 2023; Packer et al. 2023; Zhang et al. 2025; Zhong et al. 2024; Wang et al. 2023). The same persistence changes the scope of failure. A poisoned, stale, misattributed, or drifted record can be retrieved as context, support an incorrect claim, alter a tool plan and answer, and be consolidated into new memory (Xiong et al. 2026; Chao et al. 2026; Chen et al. 2025). A single local

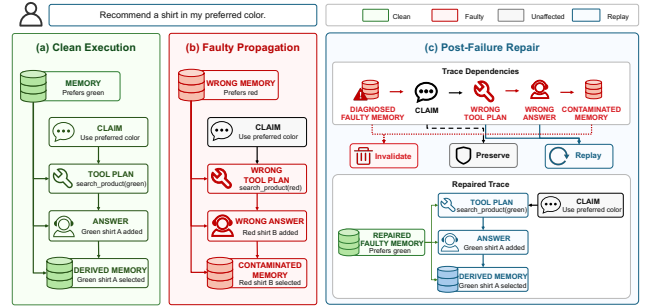


Figure 1: **A faulty memory creates downstream state.** A poisoned preference changes the claim, tool plan, answer, and a derived memory. Repair must remove unsupported descendants, preserve unaffected state, and replay the answer-relevant path.

fault can therefore become a durable family of downstream errors and reappear after the original turn.

This propagation creates the practical recovery problem illustrated in Figure 1. Correcting only the answer leaves the faulty source and contaminated descendants available to influence future turns. Deleting only the source is also insufficient after its content has been copied into claims, observations, or derived memories. At the other extreme, clearing the memory store or replaying the full trace removes valid personalization and repeats model and tool calls. Useful recovery must therefore remove unsupported consequences, retain state that still has independent evidence, and recompute only what the corrected answer requires.

Existing research largely acts at one of two boundaries. Memory defenses audit, filter, or delete suspicious records, while response-level reflection revises an unsuccessful output (Tan et al. 2026; Zou et al. 2025; Shinn et al. 2023). These interventions improve detection or local correction, but do not jointly repair the persistent store and the execution state already influenced by a fault. This gap motivates our central premise: *fault removal is not state recovery*.

We therefore formulate **post-failure memory recovery**. The input is a user session, an active memory store, a failed execution trace, and diagnosed faulty memories supplied by an upstream detector; the output is a corrected answer, re-

paired trace, and repaired active store. Diagnosis itself is outside the scope of this work, and the runtime must record the provenance connecting memory reads, claims, plans, actions, observations, answers, and mutations. Under these assumptions, the research question is precise: *once a faulty memory has been consumed, which consequences should be invalidated, which remain independently justified, and which must be regenerated?*

We propose a **dependency-guided rollback repair**. The method first constructs a typed memory-to-action graph and traces explicit downstream dataflow from each diagnosed fault. Because reachability alone would over-invalidate, it preserves candidates whose complete content or decision has independent trusted support. A deterministic planner then deletes diagnosed faults, quarantines unsupported derived state, invalidates unsupported trace outputs, and selects the answer-relevant affected computation. Finally, the executor reuses safe context and replays selected steps in trace order under the repaired store. The design directly couples the three requirements above: contamination tracing, evidence-aware preservation, and selective recomputation.

We evaluate answer recovery, state repair, preservation, and cost on 150 controlled cases across shopping, travel, and customer-support tools, covering four memory fault types, and on 50 procedural trajectories adapted from LongMemEval-V2 as a multi-fault stress test (Wu et al. 2026). Our method obtains the highest immediate recovery in both settings, although it does not achieve the lowest recurrence on the controlled benchmark. Overall, the results do not imply uniformly better trace reconstruction, but show that dependency-guided rollback repair provides a strong balance among recovery, repair cost, faulty-state cleanup, and benign-state preservation.

Our main contributions are summarized as follows:

- We formulate **post-failure memory recovery**, which jointly considers answer recovery, downstream state repair, benign-state preservation, and repair cost after faulty memories have already affected agent execution.
- We propose a **dependency-guided rollback repair** method combining explicit dataflow tracing, independent-support checking, rule-guided planning, and answer-relevant replay.
- We introduce a **150-case controlled benchmark** plus a 50-case trajectory-derived stress test, showing the highest immediate recovery and a favorable recovery–cost trade-off.

2 Related Work

Agent memory and failure detection. Persistent-memory systems support cross-session personalization and experience reuse (Park et al. 2023; Packer et al. 2023; Xu et al. 2026; Du et al. 2026; Zhao et al. 2026b). Recent work studies error propagation, hallucinated updates, stale state, retrieval or memory poisoning, and post-hoc auditing (Xiong et al. 2026; Chen et al. 2025; Chao et al. 2026; Zou et al. 2025; Chen et al. 2024; Tan et al. 2026). These methods primarily improve what enters, remains in, or is retrieved from

memory. We instead condition on diagnosed faults that have already influenced an execution.

Memory cascade repair. MemoRepair is the closest memory-side recovery framework: it withdraws invalidated descendants, rebuilds successors from retained support, and uses predecessor closure for cost-aware republication (Zhao et al. 2026a). Our setting is complementary but broader in execution scope. It jointly repairs persistent memories and a heterogeneous agent trace containing claims, plans, tool actions, observations, and the answer; answer relevance determines which invalidated execution nodes are replayed. We do not claim to replace MemoRepair’s publication contract or fault detector. Rather, we study end-to-end answer-and-state recovery once a diagnosed memory fault has crossed into agent execution.

Provenance, slicing, and rollback. Forward dependency tracing and selective recomputation draw on program and dynamic slicing (Weiser 1984; Agrawal and Horgan 1990), database provenance (Cheney, Chiticariu, and Tan 2009), and transactional rollback (Mohan et al. 1992). We do not claim graph reachability or logical undo as new. Our contribution is their adaptation to a typed memory–execution graph in which validity depends on independent evidentiary support and replay incurs model and tool cost. This scope is narrower than a new general recovery theory, but broader than repairing a memory record or final response alone.

Trace debugging and response repair. ReAct exposes reasoning and tool-use structure, while Reflexion and Self-Refine revise unsuccessful behavior or outputs (Yao et al. 2022; Shinn et al. 2023; Madaan et al. 2023). AgentTrace reconstructs causal graphs from logs for root-cause localization (Wang 2026). These ideas motivate trace-aware recovery, but localization or answer revision alone need not deactivate contaminated persistent state. Our dependency graph therefore spans the memory lifecycle and execution trace, and its rollback plan couples memory disposition with selective replay.

3 Dependency-Guided Rollback Repair

As Figure 2 shows, we propose dependency-guided rollback repair as our method. It proceeds in five stages: graph construction, affected-subgraph tracing, independent-support checking, rule-guided rollback planning, and selective replay. Rollback here means repair of agent-maintained trace and memory state. It cannot undo an irreversible external side effect. Re-executing a side-effecting tool therefore requires a resettable interface or a domain-specific compensating action. Full schemas are in Appendix A and replay prompt contracts in Appendix B.

3.1 Problem Setting

Let $\mathcal{U} = (u_1, \dots, u_k)$ be a user session, $\mathcal{M}$ the active memory store, and $\mathcal{T} = (s_1, \dots, s_n)$ a failed execution trace. A step is a memory read, claim, plan, tool action, tool observation, answer, or memory mutation. The input also contains diagnosed faulty memories $\mathcal{F} \subseteq \mathcal{M}$. At repair time, our

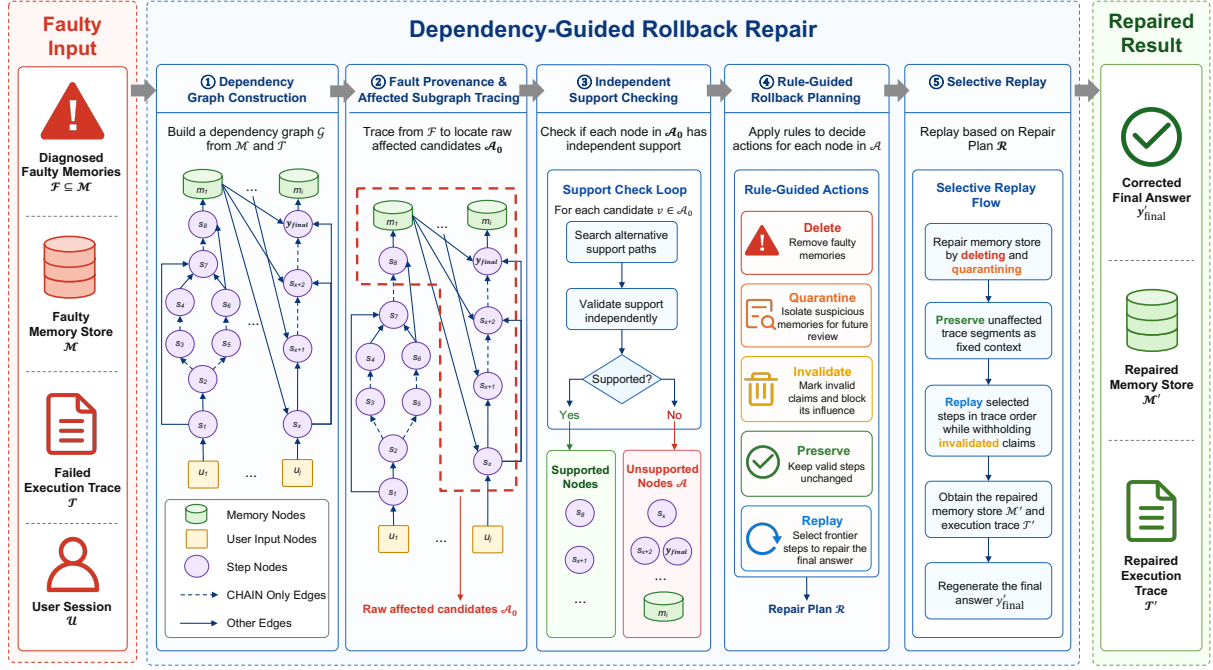


Figure 2: **Overview of dependency-guided rollback repair.** Starting from a diagnosed faulty memory set, the method constructs a memory-to-action dependency graph, traces candidate downstream effects, filters candidates with independent trusted support, generates a rule-guided rollback plan, and selectively replays answer-relevant affected computation to produce a corrected final answer and a repaired active memory store.

method sees only $(\mathcal{U}, \mathcal{M}, \mathcal{T}, \mathcal{F})$, available tools, and runtime provenance; clean traces and evaluation labels are withheld.

3.2 Dependency Graph Construction

We instantiate a directed heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ from user inputs $\mathcal{U}$, execution steps $\mathcal{T}$, and memory records $\mathcal{M}$. Construction is fault-agnostic: it records execution dataflow and memory-lifecycle structure without using $\mathcal{F}$ or any benchmark annotation. The node set is partitioned as $\mathcal{V} = \mathcal{V}_{\mathcal{U}} \cup \mathcal{V}_{\mathcal{T}} \cup \mathcal{V}_{\mathcal{M}}$, containing user inputs, typed execution steps, and persistent memory records; we identify each memory record with its node in $\mathcal{V}_{\mathcal{M}}$ when using memory sets in graph expressions.

We instantiate edges deterministically from these fields rather than inferring them. Edges point from a prerequisite or source to its dependent. They encode INITIATE, CITE, SUPPORT, PRODUCE, DELETE, UPDATE, CONSOLIDATE relations; and memory SUPERSEDE, DERIVE relations. Temporal CHAIN edges retain execution order. A node pair may carry multiple labels. Appendix A gives the full edge semantics.

Each trace record stores the identifiers of the memories, claims, observations, together with tool-call identifiers and memory-lifecycle metadata. This provenance is emitted by the agent runtime, so the method presupposes an instrumented agent. Inferring missing support or lifecycle edges from raw natural-language logs is outside the scope of this work; incomplete provenance can under-trace contamination, while spurious edges can cause unnecessary invalidation.

For propagation we use

$$\begin{aligned} \mathcal{E}_{\text{prop}} = & \mathcal{E}_{\text{initiate}} \cup \mathcal{E}_{\text{cite}} \cup \mathcal{E}_{\text{support}} \cup \mathcal{E}_{\text{produce}} \\ & \cup \mathcal{E}_{\text{delete}} \cup \mathcal{E}_{\text{update}} \cup \mathcal{E}_{\text{consolidate}} \\ & \cup \mathcal{E}_{\text{supersede}} \cup \mathcal{E}_{\text{derive}}. \end{aligned}$$

Pure CHAIN edges are excluded: occurring later is not evidence of contamination. This distinction is important because a sequential suffix may contain unrelated valid work.

3.3 Fault Provenance & Affected Subgraph Tracing

Given $\mathcal{F}$, we identify the state that may depend on these faults. This stage does not use fault-type labels such as poisoned, stale, wrong-user, or summary-drift; all diagnosed faults are handled uniformly through graph structure and memory provenance.

For each $m \in \mathcal{F}$, we use fault provenance to identify a reachability seed $r(m)$. We inspect DELETE, UPDATE, and CONSOLIDATE relations to recover lineage and identify mutations. If a failed mutation exists, it becomes the seed; otherwise, we follow a PRODUCE relation to the producing step. If m was not created during the recorded execution and has no such producing step, the memory itself becomes the seed. When $r(m)$ occurs at time t_r in the trace, only dependents timestamped no earlier than t_r are considered.

We then trace the affected subgraph from each $r(m)$ and take the union of the reached nodes and all seeds to form the

raw affected candidate set:

$$\mathcal{A}_0 = \bigcup_{m \in \mathcal{F}} \text{Reach}(\mathcal{V}, \mathcal{E}_{\text{prop}})(r(m)).$$

A node is not marked as affected merely because it follows a faulty memory in execution order; it must be connected through an explicit propagation dependency. Favoring recall, $\mathcal{A}_0$ over-approximates the truly unsupported state and is refined next.

3.4 Independent-Support Checking

A node may be reachable from a faulty memory while remaining valid because it is also justified by independent, fault-free evidence. For example, a reasoning step may consume a faulty memory together with an explicit current-turn instruction or a successful tool observation that independently supports the same conclusion. Treating every reachable node as contaminated would therefore cause unnecessary invalidation and replay.

For each semantically checkable candidate $v \in \mathcal{A}_0$ we collect its sufficient evidentiary predecessors $\text{Pred}_{\text{sup}}(v)$ through SUPPORT, DERIVE, SUPERSEDE relations and CITE relations marked sufficient by the provenance contract. Under our fixed trust policy an admissible source is an explicit current-turn instruction, an active memory not diagnosed as faulty, a successful tool observation, or an unaffected validated claim, and a source counts as independent only outside $\mathcal{F} \cup \mathcal{A}_0$: $\text{TrustedSupport}(v) \iff \exists p \in \text{Pred}_{\text{sup}}(v)$ with $p \notin \mathcal{F} \cup \mathcal{A}_0$ and $\text{Admissible}(p)$.

Diagnosed faulty memories are never eligible for preservation. Claims, plans, answers, mutations, and active derived memories are checked directly; a derived memory requires at least one sufficient provenance path terminating at it whose source and internal supporting nodes are admissible and outside $\mathcal{F} \cup \mathcal{A}_0$. Actions inherit the verdicts of their plans, and observations inherit those of their generating actions, so an independently supported plan may keep its unchanged action and observation. The check is conservative: only evidence outside $\mathcal{A}_0$ may preserve a candidate, and preserved candidates are not recursively reused to validate others, preventing mutually dependent affected nodes from validating one another. Writing $\mathcal{P}_{\text{sup}} \subseteq \mathcal{A}_0$ for the preserved candidates, including associated safe action and observation nodes, the unsupported affected set is $\mathcal{A} = \mathcal{A}_0 \setminus \mathcal{P}_{\text{sup}}$: $\mathcal{P}_{\text{sup}}$ stays eligible for reuse and $\mathcal{A}$ passes to the planner.

3.5 Rule-Guided Rollback Planning

The rollback planner converts the diagnosed faults, unsupported affected set, provenance information, and support verdicts into the repair plan $\mathcal{R}$. The planner is deterministic and rule-guided rather than optimization-based. Its memory dispositions and trace actions are orthogonal: deleting or quarantining a memory changes the active memory state, whereas invalidating, replaying, or preserving a trace node determines how the previous execution is treated.

The planner first deactivates unsafe persistent state, setting $\mathcal{M}_{\text{del}} = \mathcal{F}$ and $\mathcal{M}_{\text{quar}} = (\mathcal{A} \cap \mathcal{V}_{\text{mem}}) \setminus \mathcal{F}$, so faulty memories leave the active store, unsupported affected memories are quarantined, and support-preserved memories stay active;

when a faulty memory has a recoverable producing step, or stale state results from a failed lifecycle operation, that step stays eligible for replay. All unsupported trace nodes are then marked invalid, $\mathcal{V}_{\text{inv}} = \mathcal{A} \cap \mathcal{V}_{\text{step}}$, so their outputs cannot serve as evidence; plans pass invalidation to their actions, and actions pass it to their observations, and observations are never replayed directly, only refreshed by scheduling the generating action. Appendix C tabulates the full rule set.

To avoid replaying the full affected subgraph, the planner next identifies the answer-relevant affected computation. Let v_{final} denote the final-answer node, and let

$$\mathcal{G}^- = \mathcal{G}_{\text{dep}}[\mathcal{V} \setminus (\mathcal{M}_{\text{del}} \cup \mathcal{M}_{\text{quar}})]$$

denote the dependency graph after deleted and quarantined memories are treated as severed nodes. We define

$$\mathcal{A}_{\text{ans}} = \{v \in \mathcal{A} \cap \mathcal{V}_{\text{step}} \mid v \rightsquigarrow v_{\text{final}} \text{ in } \mathcal{G}^-\}.$$

Thus, an unsupported execution node is answer-relevant only when its repaired output can still contribute to the regenerated final answer. Nodes outside $\mathcal{A}_{\text{ans}}$ remain invalidated but are not replayed.

The executable replay set is obtained by taking the execution closure of the answer-relevant affected nodes:

$$\mathcal{V}_{\text{replay}} = \text{ExecClosure}(\mathcal{A}_{\text{ans}} \cup \{v_{\text{final}}\}).$$

The execution closure recursively adds every invalidated executable prerequisite required to recompute a selected node, including affected memory reads, controlling plans, generating tool actions, producer steps, and memory mutation steps. Safe prerequisites are not replayed and are instead supplied through $\mathcal{V}_{\text{preserve}}$. When a selected node is a tool observation, the execution closure adds its generating tool action rather than the observation itself. The final answer node is always included so that the repaired execution produces a fresh answer.

We define the preserved trace context as

$$\mathcal{V}_{\text{preserve}} = \mathcal{V}_{\text{step}} \setminus (\mathcal{V}_{\text{inv}} \cup \mathcal{V}_{\text{replay}}).$$

Therefore, $\mathcal{V}_{\text{preserve}}$ contains safe prior execution outputs needed as fixed context during replay. User inputs and active memory records are supplied separately through $\mathcal{U}$ and the repaired memory store.

3.6 Selective Replay

Given the repair plan $\mathcal{R}$, the selective replay executor first applies the planned memory dispositions. The active pre-replay memory store is

$$\mathcal{M}^- = \mathcal{M} \setminus (\mathcal{M}_{\text{del}} \cup \mathcal{M}_{\text{quar}}).$$

Records in $\mathcal{M}_{\text{quar}}$ may be retained in a separate audit partition, but they are unavailable to retrieval and execution.

The executor processes $\mathcal{V}_{\text{replay}}$ in the original trace order, thereby preserving the observed execution ordering. Memory-lifecycle relations used only for fault resolution are not treated as replay-precedence constraints. Nodes in $\mathcal{V}_{\text{preserve}}$ are supplied as fixed context, while invalidated nodes outside the replay set remain excluded from subsequent execution.

Each selected step is recomputed using the current repaired memory state and the repaired prefix of the trace. The memory state is initialized as $\mathcal{M}^-$ and updated as replayed memory mutations are executed. Invalidated claims are withheld from the replay context until replacements are generated, preventing unsupported outputs from being cited by later steps. Replayed memory reads retrieve only from the current repaired store, and replayed memory mutations are applied only when they remain justified by the repaired execution.

Tool use is also replayed selectively. If the repaired plan still requires tool use, the executor issues a new action and records the resulting observation as a new trace node. Existing observations are never independently regenerated or treated as reusable when their generating actions have been invalidated. For side-effecting tools, re-execution is performed only through the resettable, idempotent, or compensating interface assumed in the problem setting.

Let $\Delta\mathcal{M}_{\text{replay}}$ denote the ordered sequence of valid memory writes, deletions, updates, and consolidations generated during replay. The final repaired active memory store is

$$\mathcal{M}' = \text{Apply}(\mathcal{M}^-, \Delta\mathcal{M}_{\text{replay}}),$$

where Apply executes the replayed memory mutations in trace order and removes records superseded by a valid delete, update or consolidation.

The final-answer node v_{final} is always regenerated from the repaired trace. Let $\mathcal{T}'$ denote the resulting execution trace and y'_{final} the content of the regenerated final answer node. The executor outputs the corrected answer y'_{final} , repaired execution trace $\mathcal{T}'$, and repaired active memory store $\mathcal{M}'$.

4 Controlled Memory-Repair Benchmark

Task Construction We create 150 tool-use cases in shopping assistance, travel booking, and customer support, drawing on task structure from common agent benchmarks (Yao et al. 2024; Deng et al. 2023; Xue et al. 2025). Each case contains a multi-turn session, memory store, tools, and paired clean and faulty traces. At repair time every method receives the faulty store and trace plus diagnosed fault identifiers; the clean trace, expected answer, benign memory labels, and affected-node annotations are evaluation-only.

Following Section 3.1, at repair time, a method receives a user session $\mathcal{U}$, memory store $\mathcal{M}$, faulty execution trace $\mathcal{T}$, diagnosed faulty memory set $\mathcal{F}$, and the available tools. For evaluation, we additionally retain a paired clean execution, an expected repaired answer, benign memory labels, and node-level annotations of affected state. The faulty trace captures downstream contamination caused by memory failures, while the clean trace serves only as an evaluation reference and is not exposed to repair methods. The prompt contracts used to instantiate agent executions are provided in Appendix B.

Fault Injection We inject four faulty types motivated by observed memory risks (Sunil et al. 2026; Zou et al. 2025; Hatalis et al. 2023; Chao et al. 2026; Chen et al. 2025; Zhang et al. 2026; Chen et al. 2024): *poisoned* records contain adversarial or corrupted values; *stale* records survive a failed

Domain	Pois.	Stale	W-user	Drift	Total
Shopping	14	10	11	13	48
Travel	12	12	12	13	49
Customer Support	16	12	11	14	53
Total	42	34	34	40	150

Table 1: **Controlled benchmark statistics by domain and primary memory fault type.** Each task is counted once under the fault type defining its benchmark stratum. Pois. and W-user denote poisoned and wrong-user faults, respectively, while Drift denotes summary-drift.

deletion, update or consolidation; *wrong-user* records are associated with another user or context; and *summary-drift* corrupts consolidation output. A case is retained only if injection changes the final answer and creates at least one downstream effect. Thus, the benchmark evaluates recovery after diagnosed faults; it does not estimate natural fault prevalence or detector accuracy. The detailed fault injection strategies are provided in Appendix D.

Table 1 summarizes the benchmark distribution across domains and primary memory fault types. Each task is counted once under the fault type defining its benchmark stratum. Of the 150 tasks, 128 contain a single diagnosed faulty memory, while 22 contain two or three diagnosed faulty memories: 8 in customer support, 7 in shopping, and 7 in travel.

Evaluation Metrics We measure (i) *recovery*, success under the deterministic task oracle; (ii) *recurrence*, the fraction of recovered cases in which the same failure reappears when re-running the last user input; (iii) removal of diagnosed faulty memories and preservation of benign memories; (iv) claim invalidation F1, computed by micro-averaging over the gold set of faulty-trace claim-step identifiers requiring invalidation or re-derivation and the set predicted for invalidation or replay; and (v) replayed-step ratio and LLM calls. Detailed metric definitions are provided in Appendix E.

5 Experiments

We evaluate all methods on the controlled benchmark of Section 4 and evaluate transfer on an adapted LongMemEval-V2 procedural subset. All methods use GPT-4o and share the same tool environment, task inputs, faulty memory store, failed execution trace, and diagnosed faulty memory identifiers; plan-based methods are executed by the same rollback executor. Additional Gemini-3.6-Flash and Qwen3.6-27B results are reported in Appendix F. All results are from a single run at temperature 0, with seed 42 for GPT-4o and Qwen3.6-27B; Gemini-3.6-Flash does not expose a seed parameter. Qualitative analyses of representative repair cases in both settings are in Appendix G.

We compare against six baselines. Implementation details are in Appendix H. **No repair** leaves the faulty execution unchanged. Memory-centric baselines are **Full memory reset**, **Delete retrieved memories**, and a **MemAudit-style** baseline adapted from post-hoc memory auditing (Tan et al. 2026), which uses an oracle-assisted diagnostic fallback only when its audit candidate set is empty. Trace-centric baselines are **LLM-judge repair**, inspired by Reflexion (Shinn et al. 2023)

Method	Recovery $\uparrow$	Recurrence $\downarrow$	Faulty removal $\uparrow$	Benign preservation $\uparrow$	Claim-inv. F1 $\uparrow$	Replay ratio $\downarrow$	LLM count $\downarrow$
No repair	0.000	—	0.000	1.000	0.000	0.000	0.00
Full memory reset	0.407	0.197	1.000	0.000	0.000	0.061	4.03
Delete retrieved memories	0.340	0.431	0.881	0.789	0.000	0.061	4.05
MemAudit-style	0.320	0.292	0.311	0.980	0.000	0.051	10.43
LLM-judge repair	0.773	0.121	1.000	1.000	0.946	0.217	9.80
AgentTrace-style	0.607	0.341	0.000	0.997	0.853	0.091	4.69
Ours	0.853	0.266	1.000	1.000	0.566	0.123	5.70

Table 2: **Main results on the controlled benchmark.** All metrics except LLM count are reported on a 0–1 scale. Recurrence is computed only over successfully recovered cases; “—” indicates that no case was recovered. Claim-inv. F1 denotes claim invalidation F1.

Variant	Recovery $\uparrow$	Recurrence $\downarrow$	Faulty removal $\uparrow$	Benign preservation $\uparrow$	Claim-inv. F1 $\uparrow$	Replay ratio $\downarrow$	LLM count $\downarrow$
Full method	0.853	0.266	1.000	1.000	0.566	0.123	5.70
w/o support check	0.880	0.265	1.000	0.986	0.507	0.154	6.51
w/o rollback planner	0.713	0.430	1.000	0.996	0.507	0.152	9.19
w/o selective replay	0.840	0.071	1.000	1.000	0.566	0.755	24.01

Table 3: **Ablation results on the controlled benchmark.** The rollback planner mainly improves recovery and recurrence, support checking improves selectivity, and selective replay reduces repair cost.

Method	Recovery $\uparrow$	Recurrence $\downarrow$	Faulty removal $\uparrow$	Benign preservation $\uparrow$	Claim-inv. F1 $\uparrow$	Replay ratio $\downarrow$	LLM count $\downarrow$
No repair	0.000	—	0.000	1.000	0.000	0.000	0.00
Full memory reset	0.060	0.000	1.000	0.000	0.000	0.062	4.84
Delete retrieved memories	0.100	0.400	0.949	0.466	0.000	0.062	4.26
MemAudit-style	0.260	0.231	0.577	0.969	0.000	0.062	16.52
LLM-judge repair	0.260	0.077	0.613	0.983	0.369	0.244	10.64
AgentTrace-style	0.540	0.037	0.000	0.997	0.603	0.123	6.98
Ours	0.680	0.059	1.000	0.993	0.669	0.179	8.22

Table 4: **Transfer results on the adapted LongMemEval-V2 subset.** The subset contains 50 externally derived procedural tasks converted into our repair schema.

and Self-Refine (Madaan et al. 2023), and **AgentTrace-style**, adapted from causal graph tracing (Wang 2026). Prompts for LLM-judge repair are provided in Appendix B.

5.1 Main Results

Table 2 shows that our method achieves the highest recovery on the controlled benchmark. Ours recovers 85.3% of cases, compared with 77.3% for LLM-judge repair and 60.7% for AgentTrace-style repair. This corresponds to absolute gains of 8.0 and 24.6 percentage points, respectively. The gap is larger against memory-centric baselines. Ours achieves more than twice the recovery of full memory reset and about 2.5 times the recovery of Delete retrieved memories and MemAudit-style repair. Ours also removes all faulty memories while preserving 100.0% of benign memories, about 1.27 times as much benign state as Delete retrieved memories. These results suggest that dependency-guided rollback repairs faulty state without broadly deleting useful memory.

Recurrence gives a more nuanced picture. Ours reduces recurrence compared with AgentTrace-style and Delete retrieved memories, but does not obtain the lowest recurrence overall. Since recurrence is computed only over recovered cases, methods with low recovery can obtain deceptively favorable recurrence values over a smaller subset. We therefore interpret recurrence together with recovery, faulty memory

removal, and preservation.

Memory-centric baselines directly edit memory state, but they either delete too much benign memory or fail to remove downstream contamination. Full memory reset removes faulty memories but destroys all benign memory. Delete retrieved memories preserves more benign state but recovers far fewer cases than Ours. And MemAudit-style repair is conservative but misses many faulty memories and derived effects. In contrast, trace-centric methods more accurately identify claim-level trace state requiring repair. LLM-judge repair and AgentTrace-style obtain higher claim invalidation F1 than Ours on the controlled benchmark. However, this does not translate into better end-to-end repair. Compared with LLM-judge repair, Ours achieves higher recovery while reducing replay ratio by 43.3% and LLM calls by 41.8%. AgentTrace-style is cheaper, but this comes with a 24.6-percentage-point absolute drop in recovery and weaker faulty memory removal. Overall, Ours offers a favorable recovery–cost trade-off: compared with LLM-judge repair, it achieves higher recovery with fewer replayed steps and LLM calls; compared with AgentTrace-style, it substantially improves recovery and faulty memory removal at the cost of additional replay and LLM calls.

5.2 Ablation Study

Table 3 shows that the rollback planner is the component most directly responsible for answer recovery. Removing it reduces recovery from 85.3% to 71.3%, increases recurrence from 26.6% to 43.0%, and lowers benign memory preservation from 100.0% to 99.6%. It also raises replay ratio from 12.3% to 15.2% and LLM calls from 5.70 to 9.19. Equivalently, the full method uses 38.0% fewer LLM calls than the variant without the rollback planner. These results indicate that rule-guided rollback planning is important not only for identifying what should be invalidated or regenerated, but also for avoiding unnecessary repair operations.

Support checking primarily improves preservation and selectivity rather than raw recovery. Removing it increases recovery from 85.3% to 88.0% and slightly reduces recurrence from 26.6% to 26.5%, but lowers benign memory preservation from 100.0% to 98.6%. It also increases replay ratio by 25.2% and LLM calls by 14.2%. Thus, support checking introduces a small recovery trade-off in exchange for preserving more independently supported state and reducing unnecessary recomputation.

Selective replay primarily controls repair cost. Without it, recovery decreases slightly from 85.3% to 84.0%, while recurrence improves from 26.6% to 7.1%. This reduction in recurrence, however, requires substantially broader replay: replay ratio rises from 12.3% to 75.5%, and LLM calls increase from 5.70 to 24.01. Relative to this broader-replay variant, selective replay reduces replay ratio by 83.7% and LLM calls by 76.3%. Selective replay is therefore an efficiency mechanism with an explicit recurrence–cost trade-off, enabling targeted repair rather than exhaustive recomputation.

5.3 Transfer to Adapted LongMemEval-V2 Subset

We evaluate an adapted subset of LongMemEval-V2 (Wu et al. 2026). The subset contains 50 externally derived procedural and navigation-style tasks converted into our repair schema (Appendix I). Unlike the controlled benchmark, this subset targets the poisoned faulty type and is predominantly multi-fault: 5 cases contain one faulty memory, while 45 contain 2–4 faults, as summarized in Table 21. We therefore treat it as an end-to-end transfer stress test on externally derived multi-fault trajectories rather than a fault-type coverage study.

Table 4 shows that our method again achieves the highest recovery, 68.0% against 54.0% for AgentTrace-style and 26.0% for LLM-judge repair. In contrast to its controlled benchmark result, our method also obtains the highest claim invalidation F1 on this subset, scoring 0.669 versus 0.603 and 0.369. These trajectories are dominated by poisoned memories on the answer path, so the claim steps requiring repair largely overlap with the answer-relevant region selected for replay. It also ties for the highest faulty memory removal and preserves 99.3% of benign memories. AgentTrace-style keeps a cost advantage and has slightly better recurrence (3.7% vs. 5.9%) and preservation (99.7% vs. 99.3%), but trails by 14.0 points in recovery; against LLM-judge repair, our method is better on every reported transfer metric while

using 73.4% of its replay ratio and 77.3% of its LLM calls. That absolute recovery falls from 85.3% to 68.0% on externally derived trajectories is the clearest evidence that the controlled results are an upper bound rather than a deployment estimate. Together with Appendix F, this suggests that our recovery–cost profile is more consistent across settings.

6 Conclusion

This paper studies post-failure memory recovery for memory-augmented agents. We propose dependency-guided rollback repair, which traces contamination through dependency graphs, computes cost-aware rollback sets, and selectively replays affected steps. We also build a controlled benchmark spanning shopping, travel, and customer support. Across both evaluations, our method achieves the highest immediate recovery and a favorable recovery–cost trade-off. Remaining controlled-set gaps in recurrence highlight a key direction for future work: improving recurrence robustness and claim-state identification without sacrificing selective repair.

References

- Agrawal, H.; and Horgan, J. R. 1990. Dynamic Program Slicing. In *Proceedings of the ACM SIGPLAN 1990 Conference on Programming Language Design and Implementation*, 246–256.
- Chao, H.; Bai, Y.; Sheng, R.; Li, T.; and Sun, Y. 2026. STALE: Can LLM Agents Know When Their Memories Are No Longer Valid? *arXiv preprint arXiv:2605.06527*.
- Chen, D.; Niu, S.; Li, K.; Liu, P.; Zheng, X.; Tang, B.; Li, X.; Xiong, F.; and Li, Z. 2025. Halumem: Evaluating hallucinations in memory systems of agents. *arXiv preprint arXiv:2511.03506*.
- Chen, Z.; Xiang, Z.; Xiao, C.; Song, D.; and Li, B. 2024. Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases. *Advances in Neural Information Processing Systems*, 37: 130185–130213.
- Cheney, J.; Chiticariu, L.; and Tan, W.-C. 2009. Provenance in Databases: Why, How, and Where. *Foundations and Trends in Databases*, 1(4): 379–474.
- Deng, X.; Gu, Y.; Zheng, B.; Chen, S.; Stevens, S.; Wang, B.; Sun, H.; and Su, Y. 2023. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36: 28091–28114.
- Du, Y.; Wang, B.; He, Y.; Liang, B.; Wang, B.; Li, Z.; Gui, L.; Pan, J. Z.; Xu, R.; and Wong, K.-F. 2026. MemGuide: intent-driven memory selection for goal-oriented multi-session LLM agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 30584–30592.
- Hatalis, K.; Christou, D.; Myers, J.; Jones, S.; Lambert, K.; Amos-Binks, A.; Dannenhauer, Z.; and Dannenhauer, D. 2023. Memory matters: The need to improve long-term memory in llm-agents. In *Proceedings of the AAAI Symposium Series*, volume 2, 277–280.
- Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhunoye, S.; Yang,

- Y.; et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36: 46534–46594.
- Mohan, C.; Haderle, D.; Lindsay, B.; Pirahesh, H.; and Schwarz, P. 1992. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Roll-backs Using Write-Ahead Logging. *ACM Transactions on Database Systems*, 17(1): 94–162.
- Packer, C.; Fang, V.; Patil, S. G.; Lin, K.; Wooders, S.; and Gonzalez, J. E. 2023. MemGPT: towards LLMs as operating systems.
- Park, J. S.; O’Brien, J.; Cai, C. J.; Morris, M. R.; Liang, P.; and Bernstein, M. S. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, 1–22.
- Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36: 8634–8652.
- Sunil, B. D.; Sinha, I.; Maheshwari, P.; Todmal, S.; Mallik, S.; and Mishra, S. 2026. Memory poisoning attack and defense on memory based LLM-agents. *arXiv preprint arXiv:2601.05504*.
- Tan, Z.; Yao, Y.; Jin, H.; Yu, W.; Wang, G.; Fan, M.; Liu, F.; Zhang, X.; Ma, D.; Yang, T.; et al. 2026. MemAudit: Post-hoc Auditing of Poisoned Agent Memory via Causal Attribution and Structural Anomaly Detection. *arXiv preprint arXiv:2605.23723*.
- Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.
- Wang, Z. G. 2026. AgentTrace: Causal Graph Tracing for Root Cause Analysis in Deployed Multi-Agent Systems. *arXiv preprint arXiv:2603.14688*.
- Weiser, M. 1984. Program Slicing. *IEEE Transactions on Software Engineering*, SE-10(4): 352–357.
- Wu, D.; Ji, Z.; Kawatkar, A.; Kwan, B.; Gu, J.-C.; Peng, N.; and Chang, K.-W. 2026. Longmemeval-v2: Evaluating long-term agent memory toward experienced colleagues. *arXiv preprint arXiv:2605.12493*.
- Xiong, Z.; Lin, Y.; Xie, W.; He, P.; Liu, Z.; Tang, J.; Lakkaraju, H.; and Xiang, Z. 2026. How memory management impacts llm agents: An empirical study of experience-following behavior. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 623–645.
- Xu, W.; Liang, Z.; Mei, K.; Gao, H.; Tan, J.; and Zhang, Y. 2026. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 38: 17577–17604.
- Xue, T.; Qi, W.; Shi, T.; Song, C. H.; Gou, B.; Song, D.; Sun, H.; and Su, Y. 2025. An illusion of progress? assessing the current state of web agents. *arXiv preprint arXiv:2504.01382*.
- Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2024. τ -Bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv preprint arXiv:2406.12045*.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.
- Zhang, D.; Lin, Y.; Wu, Z.; Sun, Y.; Li, B.; Li, D.; and Peng, H. 2026. Useful memories become faulty when continuously updated by llms. *arXiv preprint arXiv:2605.12978*.
- Zhang, Z.; Dai, Q.; Bo, X.; Ma, C.; Li, R.; Chen, X.; Zhu, J.; Dong, Z.; and Wen, J.-R. 2025. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43(6): 1–47.
- Zhao, Y.; Dai, C.; Kou, M.; and Xiu, Y. 2026a. MEMO-REPAIR: Barrier-First Cascade Repair in Agentic Memory. *arXiv preprint arXiv:2605.07242*.
- Zhao, Y.; Yuan, B.; Huang, J.; Yuan, H.; Yu, Z.; Xu, H.; Hu, L.; Shankarampeta, A.; Huang, Z.; Ni, W.; et al. 2026b. AMA-Bench: Evaluating long-horizon memory for agentic applications. *arXiv preprint arXiv:2602.22769*.
- Zhong, W.; Guo, L.; Gao, Q.; Ye, H.; and Wang, Y. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, 19724–19731.
- Zou, W.; Geng, R.; Wang, B.; and Jia, J. 2025. {PoisonedRAG}: Knowledge corruption attacks to {Retrieval-Augmented} generation of large language models. In *34th USENIX Security Symposium (USENIX Security 25)*, 3827–3844.

A Dependency-guided Rollback Repair Data Schema

We provide the data schemas used by our implementation. Each data instance contains a user session record, a memory store, an execution trace, diagnosed faulty memories, a corresponding dependency graph, and a repair plan generated during method execution. These artifacts correspond to the modules in the main paper: the graph construction module uses lineage fields, cited identifiers, generated memory identifiers, and trace predecessor/successor fields to build typed dependency edges; the rollback planner traverses the graph from diagnosed faulty memories to identify affected claims, downstream memories, and replay candidates; and the plan executor consumes the rollback plan to produce the repaired memory store and repaired trace.

The clean store is used only as an evaluation reference, the faulty store is the input to the repair method, and the repaired store is the output produced by the plan executor. These stores share the same memory record schema. Identifiers are preserved for unchanged memories.

Memory record. As shown in Table 5, each memory record contains a memory identifier, textual content, status, source metadata, lineage metadata, and an optional trust score. In our current implementation, the trust score is recorded for analysis only, not used by the repair algorithm. Future work could estimate it more robustly and integrate it into the support checking module.

Execution step. Table 6 summarizes the fields in each execution step, including the step identifier, semantic type, tex-

Field	Description
memory_id	Unique memory identifier.
task_id	Task to which the memory belongs.
content	Natural language memory content.
source	Source of the memory.
last_modified_by	User input or step that last modified the memory.
fault_type	Fault label if the memory is faulty.
created_at	Creation timestamp.
last_modified_at	Last modification timestamp.
derived_from	Ancestor memories used to derive this memory.
supersedes	Prior memory superseded by this memory.
trust_score	Trust score assigned to the memory.
status	Memory state, one of active, deleted, quarantined, or superseded.
fact_key	Structured fact key represented by the memory.
fact_value	Structured fact value represented by the memory.
entity_id	Entity associated with the fact.

Table 5: Schema of memory records in clean, faulty, and repaired memory stores. In our implementation, `source` takes values from user inputs, tool observations, agent summaries, claims, and injected faults, while `fault_type` covers poisoned, stale, wrong-user, and summary-drift memories.

Field	Description
step_id	Unique execution step identifier.
task_id	Task to which the step belongs.
step_type	Semantic step type.
content	Natural language content of the step.
prev_step_ids	Predecessor steps in the trace.
next_step_ids	Successor steps in the trace.
invalidated_memory_ids	Memories invalidated by this step.
generated_memory_ids	Memories produced by this step.
used_ids	Memory or step identifiers used by this step.
tool_name	Tool name for tool action steps.
tool_args	Tool arguments for tool action steps.
timestamp	Step timestamp.

Table 6: Schema of execution steps in clean, faulty, and repaired traces. `step_type` covers memory reads, claims, planning steps, tool actions and observations, final answers, and memory write/delete/update/consolidation operations.

tual content, cited memories or prior steps, tool metadata when applicable, and memory mutation metadata when applicable.

Dependency graph. Each graph node has a node identifier, node kind, semantic type, and metadata, and each graph edge has a source node, target node, and one or more typed labels. Detailed fields are presented in Tables 7 and 8, with edge labels listed in Table 9.

The graph is constructed from the memory store and execution trace, so memory nodes correspond to memory records, step nodes correspond to execution step records, and edge endpoints resolve to nodes in the same task instance. Chain edges are consistent with `prev_step_ids` and `next_step_ids`; citation and support edges are consistent with `used_ids`, while production edges are consistent with `generated_memory_ids`. Edge directions follow provenance and execution flow. Supporting or antecedent nodes point to dependent nodes, and earlier execution steps point to later steps.

Repair plan. Repair plans contain task identifiers and detailed instructions for the plan executor in Table 10. Memories are deleted or quarantined, while selected steps are replayed. Invalidated claims and redundant steps support state reconstruction, and suspicious steps are retained for audit rather than replayed by the default executor.

Plan fields are interpreted as disjoint action sets unless otherwise specified: a memory cannot appear in both `delete_memory_ids` and `quarantine_memory_ids`, and a trace step can-

Field	Description
node_id	Unique graph node identifier.
node_kind	Node kind, one of memory, step, or user_input.
semantic_type	Semantic category used by the repair pipeline, including memory, user_input, or one of the execution step types in Table 6.
step_type	Original trace step type for step nodes; same vocabulary as execution step step_type, and null for non-step nodes.
memory_status	Memory status for memory nodes; same vocabulary as status, and null for non-memory nodes.
raw	Original raw record used to construct the node.

Table 7: Schema of dependency graph nodes.

Field	Description
source_id	Source node identifier in the adjacency map.
target_id	Target node identifier in the adjacency map.
labels	Edge labels defined in Table 9.

Table 8: Logical schema of dependency graph edges. In the implementation, edges are stored as adjacency maps from source nodes to target nodes with a list of typed labels.

not simultaneously appear in `replay_step_ids`, `preserve_step_ids`, and `redundant_step_ids`. All identifiers in the plan must resolve to memories or trace steps in the same task instance.

B Prompt Contracts and Templates

We summarize the structured prompt contracts used by the base agent, our rollback executor, and the LLM-judge repair baseline. Because these components are implemented through structured prompting, the contracts are part of the experimental protocol. They define what information each component can access, what evidence it must cite, and what JSON outputs it must produce. Runtime prompts instantiate these reusable contracts with domain instructions, user turns, retrieved memories, prior turns, tool schemas, tool observations, repaired state artifacts, and required output schemas. We report the contracts rather than fully instantiated prompts.

Domain Prompts. The executor prepends one domain instruction to the claim, planning, response, and memory mutation prompts. Table 11 lists the controlled benchmark instructions and the separate instruction used for the adapted LongMemEval-V2 subset. These domain prompts are intentionally concise so that domain framing does not encode repair specific behavior.

Agent Prompts. The base memory-augmented agent decomposes each turn into claim generation, planning and tool use, response generation, and post-execution memory mutation. Each stage returns strict JSON with provenance identi-

Label	Meaning
chain	Step $\rightarrow$ step; execution or reasoning order.
cite	Cited node $\rightarrow$ citing step; explicit citation relation.
support	Memory $\rightarrow$ step; the memory supports a step.
produce	Mutating step $\rightarrow$ memory; the step writes a memory.
delete	Memory $\rightarrow$ memory delete step.
update	Memory $\rightarrow$ memory update step.
consolidate	Memory $\rightarrow$ memory consolidate step.
supersede	Old memory $\rightarrow$ superseding memory.
derive	Ancestor memory $\rightarrow$ derived memory.
initiate	User input $\rightarrow$ first step of the turn.

Table 9: Dependency graph edge labels, including their direction and meaning.

Field	Description
task_id	Task identifier.
delete_memory_ids	Memories selected for deletion.
quarantine_memory_ids	Memories selected for quarantine.
invalidate_claim_ids	Claims or trace steps selected for invalidation.
replay_step_ids	Trace steps selected for replay.
preserve_step_ids	Trace steps explicitly preserved.
redundant_step_ids	Steps considered redundant after planning.
suspicious_step_ids	Suspicious but non-replayed steps.

Table 10: Schema of rollback plan artifacts.

fiers, which are recorded in the trace and later used by graph construction and repair. The planning output is parsed by the executor into concrete tool action steps. Corresponding tool results are recorded as tool observation steps and made available to response generation and memory mutation.

Rollback Repair and Baseline Prompts. Replay prompts are selected by the rollback plan and instantiated over the repaired active state. They return strict JSON with evidence ids. For targeted replay, old actions or memory mutations are shown only as untrusted hints, and the model must produce replacements grounded in repaired evidence.

C Rule-Guided Rollback Decisions

Table 14 summarizes the principal repair rules applied by the rollback planner. Memory dispositions determine which records remain active, while trace actions determine which prior outputs are invalidated, replayed, or preserved.

D Fault Injection Details

We inject faulty memories into clean benchmark instances to construct post-failure memory states. Fault selection is

Domain/source	Domain prompt
Shopping	You are a helpful shopping assistant. You can help users find products, compare prices, and provide recommendations based on their preferences.
Travel	You are a helpful travel assistant. You can help users search for flights and hotels, check availability and prices, and provide recommendations based on their travel preferences.
Customer support	You are a helpful customer support assistant. You can help users look up orders, check account or ticket status, troubleshoot issues, and provide recommendations based on their support history.
Adapted LongMemEval-V2 subset	You are a helpful long-memory assistant. Use the provided memories and prior interaction history to answer procedural or navigation-style user requests. Follow the user’s stored preferences and constraints, and use available tools only when they are needed to complete the requested procedure.

Table 11: Domain instructions used to instantiate agent prompts.

Prompt	Key instruction summary	Output fields
Claim generation	Produce a concise claim for the user’s current intent from the request, retrieved memories, and prior turns; cite all supporting memory, prior turn, and current input ids.	Claim; cited evidence
Tool planning	Plan only valid tool calls. Use read-only tools for external or volatile information and side-effecting tools only under an explicit current instruction. Ground tool arguments in available evidence, avoid invented ids, repeated failed calls, and duplicates.	Tool steps; tool name; arguments; cited evidence
Response generation	Answer from the claim, memories, and tool results. The response must be grounded, concise, and faithful to observed tool outcomes: it must not invent unavailable data, ignore returned results, or report a side-effecting action as completed unless this turn’s tool result supports it.	Response; cited evidence
Memory mutation	Propose durable memory writes, updates, deletes, or consolidations. Memories encode atomic facts; tool observation memories are immutable; every operation records provenance so later repair can identify the memories and trace steps supporting each mutation.	New memories; updates; deletes; consolidations; provenance

Table 12: Prompt contracts for the base memory-augmented agent.

Prompt	Key instruction summary	Output fields
Missing action replay	Reuse the planning contract over the repaired active state to decide whether additional tool calls are still required; empty plans are valid and tool names must come from the available tool set.	Tool steps; cited evidence
Claim and answer replay	Regenerate the claim or final answer from repaired memories, repaired history, and repaired tool observations, citing only evidence visible in the repaired context.	Claim or response; cited evidence
Targeted tool action replay	Re-derive one replacement call for each superseded tool action. Old calls and observations are untrusted hints; replacements may change tool name or arguments when needed, but may not introduce extra unbound calls.	Target action id; tool name; arguments; cited evidence
Targeted memory replay	Re-derive one replacement for each superseded memory mutation step using repaired evidence while preserving the target semantic type and avoiding protected preserved memory outputs.	Target step id; content; source; trust score; cited evidence
LLM-judge repair baseline	Predict a rollback plan from compact faulty artifacts, including faulty memories, faulty trace steps, session turns, the wrong final answer, valid ids, and the executor plan contract. The prompt requires strict JSON, forbids invented ids, deletes source faulty memories, and requests a minimal dependency complete replay region.	Delete; quarantine; invalidate; replay; preserve; redundant; suspicious; reason

Table 13: Prompt contracts for rollback replay and the LLM-judge repair baseline.

manifest-driven: for each task, a manifest specifies one or more fault specifications, including the fault type, target memory or target mutation step when applicable, and optional corrupted content. The injector does not score candidate memories or run the agent. Instead, it rewrites the clean memory store and cut trace so that the replay starts from a

seeded faulty state. When multiple faults are specified for the same task, they are applied sequentially and merged into a single faulty instance.

Each manifest row contains a task identifier and an ordered list of fault specifications:

$$r = (\tau, [f_1, \dots, f_k]).$$

Here, τ denotes the task identifier, and faults are applied sequentially in the listed order. Each fault specification f_i records the fault type and any fault specific fields needed for injection, such as the target memory, target mutation step, corrupted content, or wrong-user content. Corrupted content is used for manifest specified corruptions, while wrong-user content is used only for wrong-user faults.

Poisoned. For poisoned memory faults, the manifest names a target memory produced by a memory write step. The injector replaces the clean memory content with manifest provided corrupted content when available; otherwise, it corrupts a concrete value using deterministic domain vocabularies or numeric shifts. The faulty record receives a new faulty identifier, is marked active, and keeps the original temporal position and source metadata while being stamped with injected fault provenance. The producing trace step is also rewritten so that its generated memory id and textual content are consistent with the corrupted memory.

For example, in a shopping task, a clean memory such as “The user prefers black running shoes” may be corrupted into “The user prefers red running shoes.” The corresponding memory write step is rewritten to generate the faulty memory id and the corrupted content, so the seeded trace and memory store remain consistent.

Stale. To model stale memory faults, the injector simulates a failed memory mutation in which an old memory that should have been deleted or superseded remains active. It identifies the delete, update, or consolidation step that consumed the target memory specified in the manifest. The consumed memory inputs are reactivated as stale faulty records, while the new memories that would have been produced by the mutation, together with their downstream lineage when applicable, are removed from the seeded store. The consuming step is kept in the cut trace but rewritten so that generated memories are removed and references point to the surviving stale records.

A clean trace may update a memory stating “The user prefers economy flights” into a new memory stating “The user prefers business class flights.” A stale injection reactivates the old economy flight preference and removes the new business class preference from the seeded store. During replay, this causes the agent to behave as if the user still prefers

economy flights.

Wrong-user. Wrong-user faults are injected by fabricating a new active memory from the manifest provided wrong-user content. This memory has no producing trace step and is treated as an injected root memory store fault. It is inserted at the beginning of the memory timeline, and replay starts from the first turn so that the wrong-user memory can influence subsequent execution.

For example, a travel task for the current user may receive an injected memory stating “The user prefers hotels with airport shuttle service,” even though this preference belongs to a different user. Since the injected memory has no producing step in the current trace, it is treated as a root memory store fault and can affect replay from the first turn.

Summary-drift. Summary-drift faults use the same injection mechanism as poisoned memories, but they specifically target derived memories produced by summarization or consolidation. This models an inaccurate summary or consolidation output whose content differs from the clean memory while appearing as a confident derived memory. When specified, additional derived memories can be dropped so that the seeded store reflects the drifted summary lineage.

In a customer support scenario, two clean memories may state that “the user reported defective headphones” and “the user asked about refund eligibility after returning the item.” A correct consolidation should summarize them as “The user returned defective headphones and is seeking a refund,” but summary-drift changes the derived memory to “The user wants to keep the headphones and receive a replacement.” This drifted summary may cause the agent to create the wrong support ticket or choose an incorrect resolution policy.

E Metrics Details

We evaluate each repair method at the case level and report ratio-based metrics as percentages unless otherwise stated. Memory, claim, and replay metrics use micro-aggregation: their numerators and denominators are summed across cases before the final ratio is computed, thereby avoiding equal weighting of cases containing different numbers of memories, claims, or trace steps.

Recovery. Recovery measures whether the repaired agent produces a correct final outcome according to a deterministic

Condition	Rollback rule
Diagnosed faulty memory	Delete it from the active memory store.
Unsupported affected memory	Quarantine it and exclude it from subsequent retrieval.
Candidate supported by independent evidence	Preserve the existing trace output, together with any unchanged downstream action or observation that depends on it.
Unsupported answer-relevant execution node	Invalidate the old output and replay the corresponding executable step.
Unsupported non-answer-relevant execution node	Invalidate the old output without replay and exclude it from subsequent context.
Diagnosed faulty-memory provenance seed	If a provenance seed exists, replay the step that produced the faulty memory to generate a corrected replacement.
Final-answer node	Always regenerate the final answer under the repaired memory and trace state.

Table 14: **Rule-guided rollback decisions.** Memory dispositions determine which records remain active, whereas trace decisions determine which prior outputs are preserved, invalidated, or replayed.

task oracle. For the controlled shopping, travel, and customer support tasks, a case succeeds if the final answer contains all required facts. For adapted tasks, we apply the answer evaluator from the original LongMemEval-V2 benchmark. Let N_{succ} and N_{fail} be the numbers of cases labeled as success and failure, respectively. We compute

$$\text{Recovery} = \frac{N_{\text{succ}}}{N_{\text{succ}} + N_{\text{fail}}}.$$

Recurrence. Recurrence measures whether a repaired state remains correct when it is reused and is evaluated only on cases that pass the recovery oracle. For each successfully recovered case, we instantiate a fresh agent runtime, clear its existing state, load only the active memories from the repaired memory store, and rerun the final user input. The outcome is labeled as recurrence if its answer fails the deterministic oracle. Let N_{rec} and $N_{\text{no-rec}}$ denote the numbers of decided recurrence and no-recurrence probes, respectively. We report

$$\text{Recurrence} = \frac{N_{\text{rec}}}{N_{\text{rec}} + N_{\text{no-rec}}}.$$

Because recurrence is conditioned on successful recovery, the two metrics should be interpreted jointly.

Faulty removal. Faulty removal measures the recall of source faulty memories removed by the repair method. For case c , let F_c be the set of source faulty memory ids specified by the ground truth, and let D_c contain memories deactivated by the repair or selected for deletion or quarantine:

$$D_c = D_c^{\text{deactivated}} \cup D_c^{\text{delete}} \cup D_c^{\text{quarantine}}.$$

We compute the micro-aggregated score

$$\text{FaultyRemoval} = \frac{\sum_c |F_c \cap D_c|}{\sum_c |F_c|}.$$

Thus, the metric credits repair-attributable removal decisions rather than merely checking whether a memory is absent from the final active store. AgentTrace-style achieves a Faulty Removal score of 0 because it focuses exclusively on trace repair and leaves the memory store unchanged.

Benign preservation. Benign preservation measures how much non-faulty memory state survives the repair. For case c , let B_c denote the ground-truth benign memory ids. A benign memory that was active when repair began is considered preserved if either its original id or a successor reachable through the memory id remapping chain remains active after repair. A benign memory that was already inactive is considered preserved as long as the repair does not target it for deactivation, deletion, or quarantine. If $P_c \subseteq B_c$ is the set of preserved benign memories, we compute

$$\text{BenignPreservation} = \frac{\sum_c |P_c|}{\sum_c |B_c|}.$$

Claim-invalidating F1 ($\uparrow$). Claim-invalidating F1 evaluates whether the repair invalidates exactly the trace claims affected by the fault. For case c , let G_c be the ground-truth set of affected trace steps whose type is `claim`, and let I_c

be the set of claim ids actually invalidated by the repair. We first micro-aggregate

$$\text{TP} = \sum_c |G_c \cap I_c|, \quad \text{FP} = \sum_c |I_c \setminus G_c|, \quad \text{FN} = \sum_c |G_c \setminus I_c|,$$

and then report

$$\text{ClaimInvF1} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}.$$

The score for memory-centric baselines is always 0 because they only focus on memory store repair and no claim steps are invalidated at all.

Replay ratio. Replay ratio measures the fraction of the original trace that is actually regenerated. Let S_c be the total number of steps in the original trace and R_c the number of original target steps that are both selected for replay and successfully mapped to generated replacements. We report

$$\text{ReplayRatio} = \frac{\sum_c R_c}{\sum_c S_c}.$$

Auxiliary replacement steps generated as a consequence of replay, such as tool observations, do not increase the numerator unless they are themselves original replay targets. The metric therefore measures the selectively replayed portion of the original execution rather than the total length of the repaired trace. Although memory-centric methods modify only the memory store, they replay the final user turn to regenerate the response under the repaired memory state, resulting in a nonzero replay ratio. MemAudit-style has a lower ratio because it emits a no-op and skips final turn replay when no candidate memory is found.

LLM count. LLM count reports the average number of LLM calls made by the repair procedure per case. For our method, the per-case count is

$$L_c = L_c^{\text{decision}} + L_c^{\text{replay}},$$

where the two terms count calls used to make repair decisions and calls used to regenerate replayed steps, respectively. For N evaluated cases, we report

$$\text{LLMCount} = \frac{1}{N} \sum_{c=1}^N L_c.$$

This count excludes calls made during the original faulty execution, deterministic evaluation, and subsequent recurrence probing.

F Additional Results

This appendix reports additional controlled benchmark results. We focus on slices that explain where recovery remains difficult, how repair behavior changes across fault types, how results vary across LLM backbones, and how effectiveness relates to replay and LLM cost.

Domain	n	Recovery $\uparrow$	Recurrence $\downarrow$	Claim-inv. F1 $\uparrow$	Replay ratio $\downarrow$	LLM count $\downarrow$
Customer support	53	0.943	0.240	0.442	0.155	7.04
Shopping	48	0.729	0.400	0.727	0.076	4.31
Travel	49	0.878	0.186	0.621	0.143	5.61

Table 15: **Per-domain results for our method.** The method remains effective across all three domains. Shopping has the lowest recovery.

Fault type	n	Recovery $\uparrow$	Recurrence $\downarrow$	Claim-inv. F1 $\uparrow$	Replay ratio $\downarrow$	LLM count $\downarrow$
Poisoned	42	0.810	0.000	0.341	0.116	5.83
Stale	34	0.941	0.781	0.635	0.154	5.53
Wrong-user	34	0.706	0.125	0.421	0.152	7.12
Summary-drift	40	0.950	0.158	0.870	0.084	4.50

Table 16: **Per-fault-type results for our method.** Fault types are grouped by primary fault type. Summary-drift and stale faults have higher recovery, while wrong-user faults are harder to recover and require more LLM calls.

Backbone	Method	Recovery $\uparrow$	Recurrence $\downarrow$	Faulty removal $\uparrow$	Benign preservation $\uparrow$	Claim-inv. F1 $\uparrow$	Replay ratio $\downarrow$	LLM count $\downarrow$
GPT-4o	LLM-judge repair	0.773	0.121	1.000	1.000	0.946	0.217	9.80
GPT-4o	AgentTrace-style	0.607	0.341	0.000	0.997	0.853	0.091	4.69
GPT-4o	Ours	0.853	0.266	1.000	1.000	0.566	0.123	5.70
Gemini-3.6-Flash	LLM-judge repair	0.740	0.108	1.000	0.987	0.604	0.183	8.37
Gemini-3.6-Flash	AgentTrace-style	0.513	0.156	0.000	0.997	0.853	0.091	4.64
Gemini-3.6-Flash	Ours	0.733	0.091	1.000	1.000	0.566	0.123	5.53
Qwen3.6-27B	LLM-judge repair	0.613	0.196	1.000	0.993	0.739	0.133	6.37
Qwen3.6-27B	AgentTrace-style	0.487	0.260	0.000	0.997	0.853	0.091	4.64
Qwen3.6-27B	Ours	0.660	0.061	1.000	1.000	0.566	0.123	5.52

Table 17: **Backbone sensitivity on the controlled benchmark.** Supplemental Gemini-3.6-Flash and Qwen3.6-27B runs evaluate Ours together with the closest LLM-based and trace-based repair competitors.

Per-domain results. Table 15 shows that performance varies across domains. Customer support achieves the highest recovery at 94.3%, while shopping is the hardest domain, with recovery dropping to 72.9%. Shopping also has the highest recurrence at 40.0%, suggesting that repaired answers are more likely to remain vulnerable to downstream state errors in this domain. We attribute this difficulty to fine-grained product distinctions, including product ids, sellers, prices, inventory, and user preferences, where a small reconstruction error can change the selected item or seller.

Per-fault-type results. Table 16 groups tasks by primary fault type. Summary-drift has the highest recovery at 95.0% and the highest claim invalidation F1 at 0.870, likely because drifted summaries often have compact provenance through update or consolidation steps. Wrong-user faults are the hardest, with recovery dropping to 70.6% and the highest LLM count, reflecting that wrong-user memories can look like coherent personalized records unless the trace exposes a conflict with the current user’s history. Stale faults show a different pattern. Recovery is high at 94.1%, but recurrence is also high at 78.1%, indicating that final answer repair can succeed while stale state suppression remains fragile. Poisoned faults have the lowest claim invalidation F1, but recurrence is 0 among recovered cases, suggesting that once the corrupted value is removed or refreshed, the repaired state is less likely to reintroduce the same failure.

Backbone sensitivity. Table 17 reports supplemental controlled benchmark runs with Gemini-3.6-Flash and Qwen3.6-

27B. We run all three LLMs with temperature 0 and seed 42, except that Gemini-3.6-Flash does not accept a seed parameter. The goal is to test whether the main findings depend on a particular LLM backbone, rather than to run the full comparison across all repair paradigms. We therefore compare Ours with LLM-judge repair, the closest LLM-based competitor, and AgentTrace-style, the closest trace-based competitor. Compared with GPT-4o, recovery decreases for all three methods under the additional backbones, with the largest drops under Qwen3.6-27B. Under Gemini, LLM-judge repair slightly exceeds Ours in recovery, 0.740 versus 0.733, corresponding to only one additional recovered case. However, Ours has lower recurrence, 0.091 versus 0.108, perfect benign preservation, and lower replay and LLM cost. The lower benign preservation of LLM-judge repair under Gemini and Qwen reflects occasional over-repair, where the model removes or quarantines non-target memories. Overall, these supplemental results support the main experimental conclusions: backbone choice affects absolute recovery and repair-plan quality, but dependency-guided rollback continues to provide a stronger balance of recovery, recurrence, preservation, and cost.

Repair cost breakdown. Table 18 shows that our method is not simply buying recovery with more computation. Compared with LLM-judge repair, our method improves recovery from 77.3% to 85.3% while using less than half the total tokens and fewer LLM calls. AgentTrace-style is cheaper, with a lower replay ratio of 9.1%, but its recovery drops to 60.7%, showing the cost of under-repairing persistent memory con-

tamination. The ablations clarify the main cost driver. Removing selective replay raises the replay ratio from 12.3% to 75.5%, while recovery does not improve over the full method. Removing support checking mainly affects selectivity, and removing the rollback planner hurts both recovery and efficiency.

Sensitivity to fault and memory scale. Table 19 reports robustness slices by faulty memory count and benign active memory store size. These are not controlled scaling laws because bucket sizes and fault compositions vary, but the method does not collapse on the available multi-fault cases. Recovery is 85.2% for one-fault cases and 82.4% for two-fault cases, while the three-fault bucket remains recoverable but contains only five cases. Memory store size is also non-monotonic. Recovery drops to 77.8% in the 6–8 bucket but rises to 91.4% in the 9–11 bucket. Replay ratio remains modest across buckets, ranging from 10.8% to 15.5%. These trends suggest that repair difficulty depends more on dependency shape, fault semantics, and whether the faulty memory lies on a load-bearing answer path than on memory store size alone.

G Case Study

We present one running example, two compact cases from the controlled benchmark, and one multi-fault case from the adapted LongMemEval-V2 subset. The customer support example illustrates the full repair pipeline, while the travel and shopping cases highlight two complementary mechanisms: cleaning a wrong-user root memory together with contaminated derived memories, and replaying a load-bearing tool-call path after a poisoned factual memory. The final case illustrates how multiple poisoned memories can reinforce the same error across different levels of abstraction, requiring both persistent memory cleanup and trace-level replay for complete repair.

G.1 Running Example: Customer Support Summary-Drift Case

We use `customer_support_task_036` because the fault propagates beyond the final answer into downstream claims and memory mutations.

Task context.

User input: Using only the latest concise support case summary you stored, tell me the order status and refund window together.

Faulty answer: The order status for ORD00169 is shipped, and the refund window is 45 days.

Expected repaired answer: Order ORD00169 is shipped, and the Home refund window is 30 days.

Fault provenance. The failure is caused by a drifted memory update. The prior summary `m_019` correctly records that order ORD00169 is shipped and has a Home refund window of 30 days. The faulty updated summary `m_f026` changes the refund window to 45 days. Given diagnosed faulty memory `m_f026`, the provenance module follows the

incoming `PRODUCE` edge to update step `s_132`, which consumed `m_019`, invalidated it, and produced the drifted summary. This identifies the fault as summary-drift during memory update rather than an isolated root memory store fault.

Affected state and support checking. Forward tracing from `s_132` and `m_f026` identifies 21 affected nodes: 5 memory nodes, 4 memory read steps, 4 claim steps, 4 final answer steps, 3 memory write steps, and 1 memory update step. The answer-relevant region contains the faulty final answer `s_148`, which depends on a downstream claim using the drifted refund window.

Support checking prevents all reachable nodes from being treated as unsafe. After removing the faulty summary, the checker marks `s_132` as independently supported, while downstream affected claims and answer steps remain unsupported. It also separates affected but supported derived memories from unsafe ones, preserving `m_027`, `m_028`, `m_029`, and `m_030` because they remain independently supported despite being reachable from the faulty summary.

Rollback and replay. The planner deletes `m_f026`, replays the faulty summary producer and the answer-relevant downstream computation, and leaves unsupported but not load-bearing steps suspicious rather than replaying them. Support verdicts are therefore not hard constraints. `s_132` is replayed because it produced the faulty summary, even though its source evidence can be independently supported. Conversely, `s_137` is left suspicious because it is unsupported but not load-bearing for the corrected final answer.

After replay, the faulty summary is replaced by corrected memory `m_n034`, which states that the Home refund window is 30 days. Unrelated support memories are preserved, `m_028` remains active, and affected side memories are either preserved or regenerated according to their support status.

Corrected output and baseline behavior. The repaired final answer states that order ORD00169 is shipped and that the Home refund window is 30 days. No repair propagates the drifted summary. MemAudit-style preserves the drifted summary because memory auditing alone does not force replay of the affected summary and answer path. Delete retrieved memories removes useful context and yields an incomplete answer, while full reset deletes the support history needed to answer from memory. LLM-judge repair and AgentTrace-style recover the answer, but do not produce the same selective rollback account over the faulty producer and downstream state.

G.2 Derived Faulty Memories: Travel Wrong-User Memory Case

Instance `travel_task_048` asks for the airline of the exact confirmed active flight. The faulty answer is Qantas, while the expected repaired answer is ANA. The source fault is a wrong-user memory `m_f000` stating that the active flight airline was corrected to Qantas rather than ANA. Since this memory has no producing step in the current trace, our method treats it as a root memory store fault.

Method	Recovery $\uparrow$	Total tokens $\downarrow$	LLM count $\downarrow$	Replay ratio $\downarrow$
No repair	0.000	0.00	0.00	0.000
Full memory reset	0.407	6509.33	4.03	0.061
Delete retrieved memories	0.340	7399.60	4.05	0.061
MemAudit-style	0.320	19700.95	10.43	0.051
LLM-judge repair	0.773	28175.28	9.80	0.217
AgentTrace-style	0.607	10508.94	4.69	0.091
Ours	0.853	12697.55	5.70	0.123
w/o support check	0.880	12954.37	6.51	0.154
w/o rollback planner	0.713	14969.29	9.19	0.152
w/o selective replay	0.840	55018.01	24.01	0.755

Table 18: **Repair cost breakdown.** Our method achieves higher recovery than LLM-judge repair while using fewer tokens and fewer LLM calls. The ablations show that selective replay is the main mechanism preventing repair cost from increasing sharply.

Slice	Bucket	n	Recovery $\uparrow$	Recurrence $\downarrow$	Claim-inv. F1 $\uparrow$	Replay ratio $\downarrow$	Total tokens $\downarrow$	LLM count $\downarrow$
Faulty memories	1	128	0.852	0.266	0.553	0.119	12559.85	5.68
Faulty memories	2	17	0.824	0.357	0.622	0.135	12586.29	5.59
Faulty memories	3	5	1.000	0.000	0.714	0.155	16601.00	6.60
Memory store size	0–5	27	0.889	0.583	0.617	0.149	8806.26	5.37
Memory store size	6–8	36	0.778	0.071	0.435	0.131	15304.03	6.50
Memory store size	9–11	58	0.914	0.321	0.575	0.110	12465.07	5.69
Memory store size	≥ 12	29	0.793	0.043	0.667	0.108	13549.83	5.03

Table 19: **Robustness slices by faulty memory count and memory store size.** Memory store size is measured by the number of benign active memories. These slices are intended as robustness checks rather than scaling claims, since bucket sizes and fault compositions are not controlled independently.

The wrong-user memory also contaminates a downstream active flight summary. In the clean store, `m_008` records flight `F0028` with airline `ANA` and price `$275.99`. In the faulty run, the derived summary is rewritten with `Qantas`. Thus the final answer is supported both by the wrong-user root memory and by a contaminated derived memory. Our method deletes `m_f000`, remaps the contaminated active flight summary to corrected memory `m_n012`, and recovers `ANA` while preserving unrelated travel memories.

This case illustrates why persistent memory state repair is necessary. No repair and AgentTrace-style leave the root wrong-user memory and derived summary active, MemAudit-style removes the derived summary but misses the root memory, and LLM-judge repair fixes the immediate answer but misses the contaminated derived memory, leading to recurrence. Full reset and delete retrieved memories can recover the answer, but only by sacrificing substantially more benign state. Delete retrieved memories preserves only 6 in 10 benign memories, while full reset deletes all 12 active memories.

G.3 Tool Call Replay: Shopping Poisoned Memory Case

Instance `shopping_task_014` asks the agent to compare current prices for the remembered product. The faulty answer selects `StoreB` at `$66.95` and the expected repaired answer selects `StoreC` at `$63.99`. The source fault `m_f003` poisons the `StoreC` price for product `P0199`, raising it to `$88.67`. This makes the faulty answer locally consistent with corrupted price evidence, since `StoreB` appears cheaper than `StoreC`.

Reliable repair therefore requires more than deleting the poisoned memory or editing the final response. The load-bearing comparison path must be replayed so that `compare_price(P0199)` refreshes the tool-derived evidence. Our method deletes `m_f003`, replays the answer-relevant path including the comparison tool call, regenerates the price comparison memory, and answers `StoreC` at `$63.99`. The replayed tool result reports `StoreC` at `$63.99`, `StoreA` at `$67.00`, and `StoreB` at `$66.95`, which flips the final comparison while preserving other active product memories.

All baselines fail this case. Memory-centric methods either leave the poisoned price active or remove too much product context. LLM-judge repair deletes the poisoned memory, and AgentTrace-style avoids keeping it active, but neither recovers the corrected tool-derived evidence needed to change the final comparison. The case shows why dependency-guided rollback must sometimes replay the tool action that produced or refreshed contaminated evidence.

G.4 Multi-Fault: Adapted LongMemEval-V2 Case

We include one adapted LongMemEval-V2 case to illustrate how multiple poisoned memories can jointly contaminate execution. Instance `lme_task_025_901b7d17` asks the agent to identify the new header action button that appears after opening a ServiceNow knowledge article from the search-results page. The expected answer is `Attach to Private Task`, while the faulty execution answers `Add to Private Task`.

This case contains three poisoned memories that en-

code the same wrong conclusion at different abstraction levels. Memory `m_f005` corrupts the retrieved evidence state by recording the article-page header button as `Add to Private Task`. Memory `m_f006` turns this corrupted observation into a derived factual statement, claiming that `Add to Private Task` is the label visible near the article heading and absent from the search-results context. Memory `m_f009` further consolidates the error into a compact answer memory. The new article-page header action label is `Add to Private Task`. Thus the final answer is not caused by a single isolated poisoned fact. The same wrong label is reinforced across raw evidence, extracted observation, and compact answer memory.

Starting from the diagnosed faulty memories, our method deletes `m_f005`, `m_f006`, and `m_f009`. During rollback, it invalidates the affected claim steps `s_046` and `s_049`, replays the answer-relevant path, and reruns the evidence lookup needed to refresh the article-page observation. The repaired execution generates replacement memories `m_n010`, `m_n011`, and `m_n012`. These replacements consistently record the corrected label `Attach to Private Task`: the repaired raw evidence memory stores the corrected article-page button, the repaired derived observation states that this label appears on the article page, and the repaired compact answer memory identifies it as the new header action. The corrected final answer is therefore `Attach to Private Task`.

This case highlights why multi-level poisoned memories require both memory-state cleanup and trace-level replay. No repair keeps the poisoned memories active and repeats the faulty answer. Memory-centric baselines delete the poisoned memories but fail to recover the answer. MemAudit-style answers `Subscribe`, while full reset and delete retrieved memories answer `Edit` after losing useful context. LLM-judge repair also deletes all three poisoned memories but still regenerates `Add to Private Task`. AgentTrace-style recovers the final answer through trace-level replay, but without the same explicit cleanup account over all jointly poisoned memory levels. Our method recovers the answer while deleting all three poisoned memories, preserving all benign memories, and regenerating consistent replacement memories for the raw evidence, derived observation, and compact answer.

H Baseline Implementation Details

We summarize the baseline implementations used in our evaluation. Since prior methods do not directly target post-failure repair for memory-augmented executions, we adapt each method to the same repair interface. Each method receives the failed trace, faulty memory store, diagnosed faulty memory ids, and final turn task input, then outputs either a memory edit decision or a rollback plan executed by the shared replay executor. Table 20 summarizes the configurations.

No Repair. This lower-bound condition keeps the failed run unchanged, including the faulty memory store, faulty trace, and original final answer. It emits an empty repair plan and performs no cleanup, deletion, rollback, or replay.

Full Memory Reset. This memory-centric baseline removes all active memories before replaying the final user turn under the same agent lifecycle. The trace prefix before the final turn is kept fixed, and the regenerated final turn trace is normalized to the same artifact schema as other methods. This strategy removes all potentially contaminated memory, but also deletes benign personalization state.

Delete Retrieved Memories. This baseline deletes active memories that appear in the affected region of the failed execution before the final turn. Starting from the affected subgraph seeded by the diagnosed faulty memories, it collects active memories retrieved or referenced by relevant non-retrieval steps, where non-retrieval steps refer to all steps except memory read steps, then deletes those memories and replays the final user turn. It does not perform independent-support checking, derived memory cleanup, or answer-relevant rollback optimization.

LLM-Judge Repair. This baseline asks an LLM to predict a rollback plan from a compact view of the failed execution, including session turns, diagnosed faulty memory ids, compact memory records, compact trace steps, the wrong final answer, valid id sets, and the executor output schema. The output is constrained to a JSON rollback plan with fields for memory deletion, memory quarantine, replay steps, preserved steps, and suspicious steps. The plan must reference only provided memory and step identifiers and is required to delete every diagnosed faulty memory.

We validate each generated plan before execution. Plans are rejected if they contain invalid JSON, malformed fields, invalid ids, overlapping step dispositions, or missing deletion of diagnosed faulty memories. Validation failures are retried with the error appended to the prompt. If no valid plan is produced within the retry budget, the case is marked as failed. Valid plans are executed by the same rollback executor as our method, isolating the effect of the planning strategy.

MemAudit-style. We adapt MemAudit-style post-hoc memory auditing (Tan et al. 2026) as a memory deletion selector. The candidate pool contains active memories before the final user turn that are used by downstream non-retrieval steps. If this pool is empty, we fall back to diagnosed source fault memories that remain active. We report this fallback as an oracle-assisted diagnostic fallback used only to avoid an empty audit candidate set, not as an unsupervised MemAudit signal. Candidate memories are scored using a memory consistency anomaly signal and a counterfactual influence signal. The consistency graph compares memories with matching entity and fact fields. Matching values induce support edges, while different values induce contradiction edges.

For candidate memory m , the anomaly score is

$$\text{anomaly}(m) = \frac{\text{contradiction}(m)}{\text{support}(m) + \text{contradiction}(m)},$$

with score 0 when the denominator is zero. In the oracle-assisted diagnostic variant, if benchmark provided faulty content directly matches a memory, that memory is assigned anomaly score 1.0. Counterfactual influence is computed by

Method	Repair level	Memory repair	Trace rollback	Dependency graph
No repair	None	No	No	No
Full memory reset	Memory	All memories	No	No
Delete retrieved memories	Memory	Retrieved memories	No	No
MemAudit-style	Memory	Ranked memories	No	Memory only
LLM-judge repair	Trace	LLM-selected	LLM-selected	No
AgentTrace-style	Trace	Indirect	Suspicious steps	Trace only
Ours	Memory+trace	Selective	Selective	Memory+trace

Table 20: **Comparison of repair baselines.** Baselines differ in whether they repair memory state, roll back faulty trace state, or use dependency structure.

deleting m , rerunning the final user turn, and scoring the regenerated trace with the deterministic task oracle:

$$\text{influence}(m) = \frac{\max(0, S_{-m} - S_{\text{orig}})}{1 - S_{\text{orig}}},$$

where S_{orig} is the original faulty trace score and S_{-m} is the score after deleting m . If the original score is already 1, or if the rerun fails, the influence score is set to 0. The final score is

$$\text{score}(m) = \alpha \cdot \text{influence}(m) + (1 - \alpha) \cdot \text{anomaly}(m),$$

with $\alpha = 0.5$. Memories with score above 0.5 are deleted together, and only the final user turn is replayed. This baseline does not roll back trace nodes, invalidate claims, repair derived memory lineage, or select answer relevant replay frontiers.

AgentTrace-style. We adapt AgentTrace-style causal trace localization (Wang 2026) as a trace-centric repair baseline. The method receives the same diagnosed faulty memory ids as other methods, constructs faulty memory provenance and affected regions, scores candidate trace steps, selects the highest ranked step as the root cause, and converts it into a replay plan executed by the shared rollback executor. We use this adaptation only to choose replay regions, not to implement a full persistent memory repair workflow.

Candidate steps are restricted to memory read, claim, plan, tool action, and final answer steps that are backward reachable from the erroneous final answer and fault-relevant. A step is fault-relevant if it lies in the affected trace region, directly uses faulty memory, belongs to the fault dependent suffix, or is the final answer step. Each candidate v receives

$$r(v) = 0.40f_{\text{back}}(v) + 0.25f_{\text{aff}}(v) + 0.20f_{\text{fault}}(v) + 0.10f_{\text{down}}(v) + 0.05f_{\text{type}}(v),$$

where $f_{\text{back}} = 1 - d/d_{\text{max}}$ measures backward relevance, f_{aff} and f_{fault} indicate affected region membership and direct faulty memory involvement, $f_{\text{down}} = \min(1, \text{reached_steps}/8)$ measures downstream reachability and is raised to at least 0.5 if the step reaches the erroneous final answer, and f_{type} is a fixed step type prior set to 0.95, 0.85, 0.75, 0.65, and 0.40 for memory read, claim, plan, tool action, and final answer steps, respectively. The weights and priors are fixed across all domains and fault types.

If no valid candidate is found, the baseline falls back to the final answer step. The replay region combines provenance source steps for diagnosed faulty memories with the

sequential suffix from the selected root cause step through the erroneous final answer. Persistent memory cleanup is disabled, so existing faulty memories remain active unless overwritten as a side effect of replay.

I Adapted LongMemEval-V2 Subset Details

Subset Construction. We adapt a subset of LongMemEval-V2 (Wu et al. 2026), which consists of long-context user-assistant interaction trajectories. We do not use the full benchmark distribution. Instead, we retain only cases for which our pipeline can construct reliable clean data for memory repair evaluation. Specifically, the trajectory must contain sufficient textual evidence to recover the clean answer, the evidence must be convertible into our controlled benchmark schema, and the clean answer must be verifiable without relying on hidden gold labels.

This selection criterion favors procedural and navigation-style tasks, whose answers are grounded in explicit trajectory evidence such as page titles, menu labels, form fields, or ordered action sequences. We exclude cases that cannot be reliably converted into clean instances, including ambiguous boolean questions, generic short-answer questions, visual-only UI states, and aggregation questions whose required evidence is not explicit in the retained trajectory text. The resulting adapted set contains 50 cases.

Schema Adaptation. Each selected trajectory is converted into the same schema used by our controlled benchmark. Trajectory observations, action traces, page states, and supporting evidence snippets are mapped into memory records with provenance metadata. Fault construction follows the poisoned memory procedure described in Appendix D.

Final answer evaluation follows the same clean answer validation protocol as our main benchmark. When an answer contains multiple fields or ordered steps, we evaluate it using claim-level checks derived from the clean trajectory evidence.

Fault Distribution. Table 21 shows the number of faulty memories per adapted case. Most cases in this subset contain multiple faulty memories, complementing our controlled benchmark, where most cases contain a single faulty memory.

Limitations. The adapted LongMemEval-V2 subset is intended as a transfer stress test rather than a replacement for the full LongMemEval-V2 benchmark. Because we retain only cases from which reliable clean data can be constructed

Number of faulty memories	Number of cases
1	5
2	5
3	38
4	2

Table 21: Fault multiplicity in the adapted LongMemEval-V2 subset.

under our repair schema, the subset does not represent the full LongMemEval-V2 distribution and should not be used to claim general LongMemEval-V2 performance. Its purpose is to test whether repair behavior transfers from our controlled benchmark to trajectory-derived memory records.